

GitHub Source Code Link:

https://github.com/chirayus20/Jenkins_CI_CD_Chirayu

Part 1: Deploy Flask and Express on a Single EC2 Instance

Objective: Deploy both the Flask backend and the Express frontend on a single Amazon EC2 instance.

Provisioning the EC2 Instance

1. Launch an EC2 instance on AWS (a free-tier eligible instance is fine).
2. SSH into the instance and install the dependencies: Python (for Flask), Node.js (for Express), and Git (for pulling the application code).

Application Setup

3. Clone the Flask and Express repositories onto the EC2 instance.
4. Install the required dependencies for both applications using `pip` and `npm`.
5. Configure both applications to run on different ports (e.g., Flask on port 5000 and Express on port 3000).
6. Start the applications using process managers like `pm2` or `systemd` to ensure they remain active.

Deliverables

7. A running EC2 instance with Flask and Express accessible via the instance's public IP.
8. A description or diagram of the deployed architecture.

Part 2: Implement CI/CD Pipeline Using Jenkins

Objective: Automate the deployment of the Flask and Express applications using Jenkins.

Install Jenkins

9. Install Jenkins on the same EC2 instance or on a separate machine.
10. **Configure Jenkins** by installing essential plugins like Git, NodeJS, and Python.

Set Up Jenkins Pipeline

11. Create two separate Jenkins pipelines for the Flask and Express applications.
12. **Pipeline Steps:** Pull the latest code from the respective Git repositories.
13. Install dependencies for Flask (`pip install -r requirements.txt`) and Express (`npm install`).
14. Restart the applications using the process manager (e.g., `pm2 restart`).

Triggering the Pipeline

15. Set up a GitHub webhook to trigger the Jenkins pipeline on every push to the repositories.

Optional Enhancements

16. Add testing stages to the pipeline for both applications.
17. Configure environment variables in Jenkins for managing sensitive data like API keys.

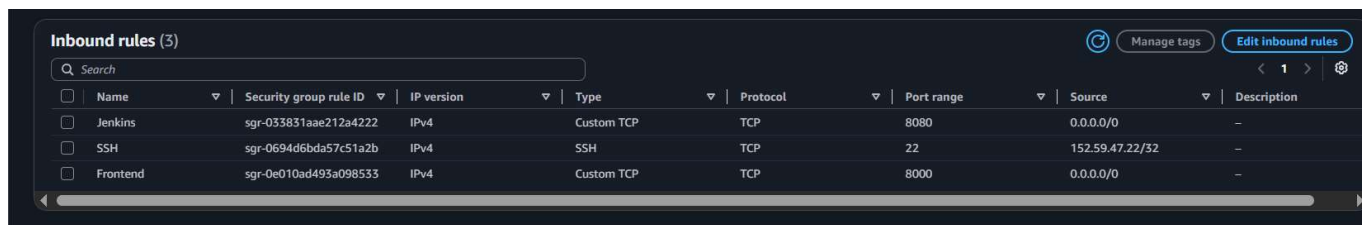
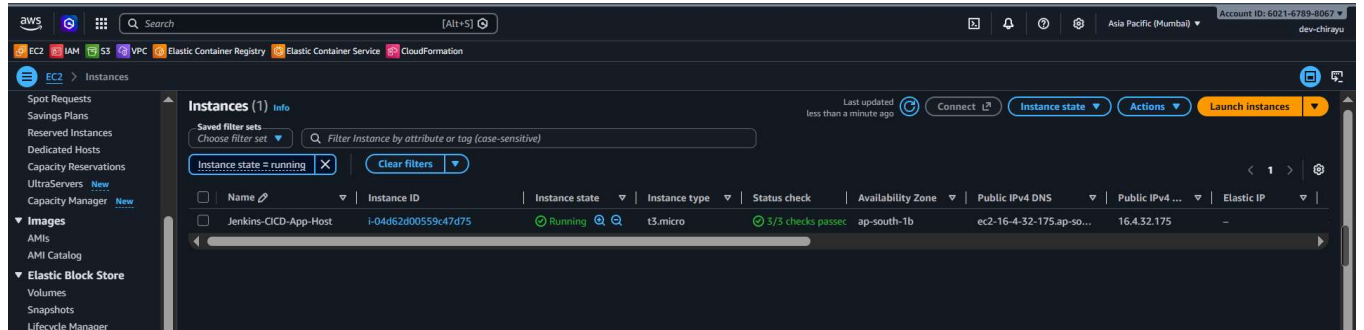
Deliverables

18. A fully functional CI/CD pipeline that automates the deployment process for Flask and Express.
19. A Jenkins pipeline script (`Jenkinsfile`) for each application.
20. Evidence of the pipeline working (e.g., screenshots of successful builds and deployments).

Part 1: Deploy Flask and Express on a Single EC2 Instance.

Explanation: Part 1

Step 1: EC2 Instance Launch & Security Group Configuration.



Step 2: Server Dependencies Installation.

We will SSH into the EC2 instance and run the following commands:

1. Update system packages

```
sudo apt update && sudo apt upgrade -y
```

2. Install Git and build tools

```
sudo apt install -y git curl wget build-essential
```

3. Install Python 3 and pip (Flask backend)

```
sudo apt install -y python3 python3-pip python3-venv
```

4. Install Node.js and PM2 (Express frontend)

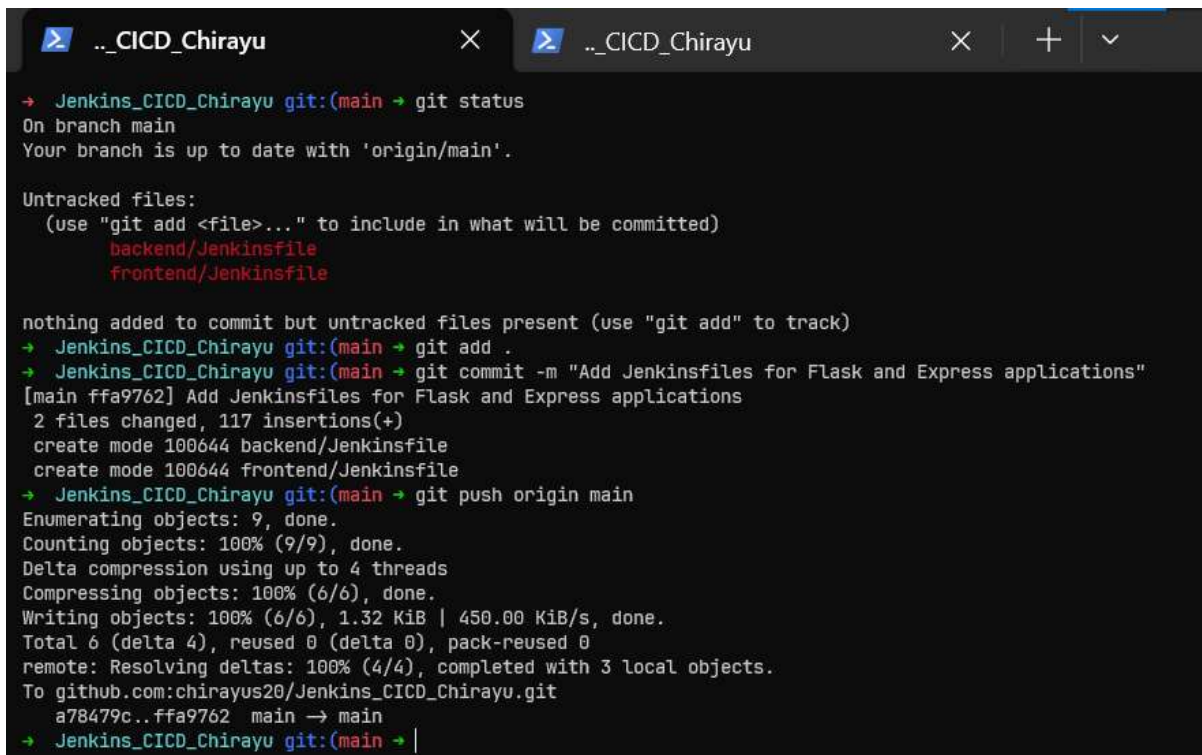
```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

```
sudo npm install -g pm2
```

Step 3:

- Launched a t3.micro Ubuntu EC2 instance on AWS, connected over SSH, and installed Python, Node.js, Git, and PM2.
- Cloned the repository, installed dependencies with pip (in a virtual environment) and npm, and configured Flask on port 9000 and Express on port 8000.
- Started both apps with PM2 so they stay running, and opened ports 8000 (Express) and 9000 (Flask) in the Security Group to reach them at <http://13.203.220.158:8000/>



```
→ Jenkins_CICD_Chirayu git:(main → git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        backend/Jenkinsfile
        frontend/Jenkinsfile

nothing added to commit but untracked files present (use "git add" to track)
→ Jenkins_CICD_Chirayu git:(main → git add .
→ Jenkins_CICD_Chirayu git:(main → git commit -m "Add Jenkinsfiles for Flask and Express applications"
[main ffa9762] Add Jenkinsfiles for Flask and Express applications
 2 files changed, 117 insertions(+)
 create mode 100644 backend/Jenkinsfile
 create mode 100644 frontend/Jenkinsfile
→ Jenkins_CICD_Chirayu git:(main → git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.32 KiB | 450.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To github.com:chirayus20/Jenkins_CICD_Chirayu.git
 a78479c..ffa9762  main → main
→ Jenkins_CICD_Chirayu git:(main → |
```

Part 2: Implement CI/CD Pipeline Using Jenkins

5. Install Java and Jenkins

```
sudo apt install -y fontconfig openjdk-21-jre
```

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc  
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]  
https://pkg.jenkins.io/debian-stable binary/" | sudo tee  
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt update
```

```
sudo apt install -y jenkins
```

6. Start Jenkins

```
sudo systemctl enable jenkins
```

```
sudo systemctl start jenkins
```

7. Initial admin password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

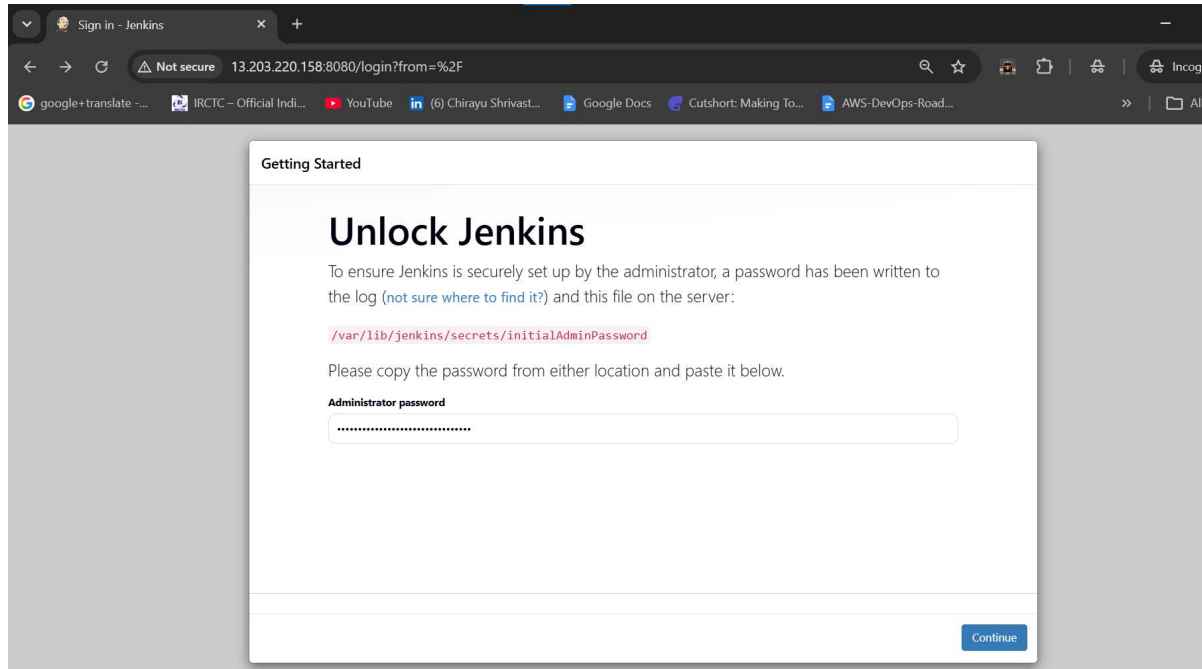
Step 3: Jenkins is active (running) perfectly. Sudo access has also been set up.

```
ubuntu@ip-172-31-2-191: ~/Je x  _CICD_Chirayu x + v
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu$ echo "jenkins ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/jenkins
jenkins ALL=(ALL) NOPASSWD: ALL
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu$ sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 07:33:46 UTC; 2h 53min ago
     Main PID: 35167 (java)
       Tasks: 37 (Limit: 1082)
      Memory: 267.9M (peak: 317.0M swap: 111.6M swap peak: 136.6M)
         CPU: 41.948s
    CGroup: /system.slice/jenkins.service
            └─35167 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

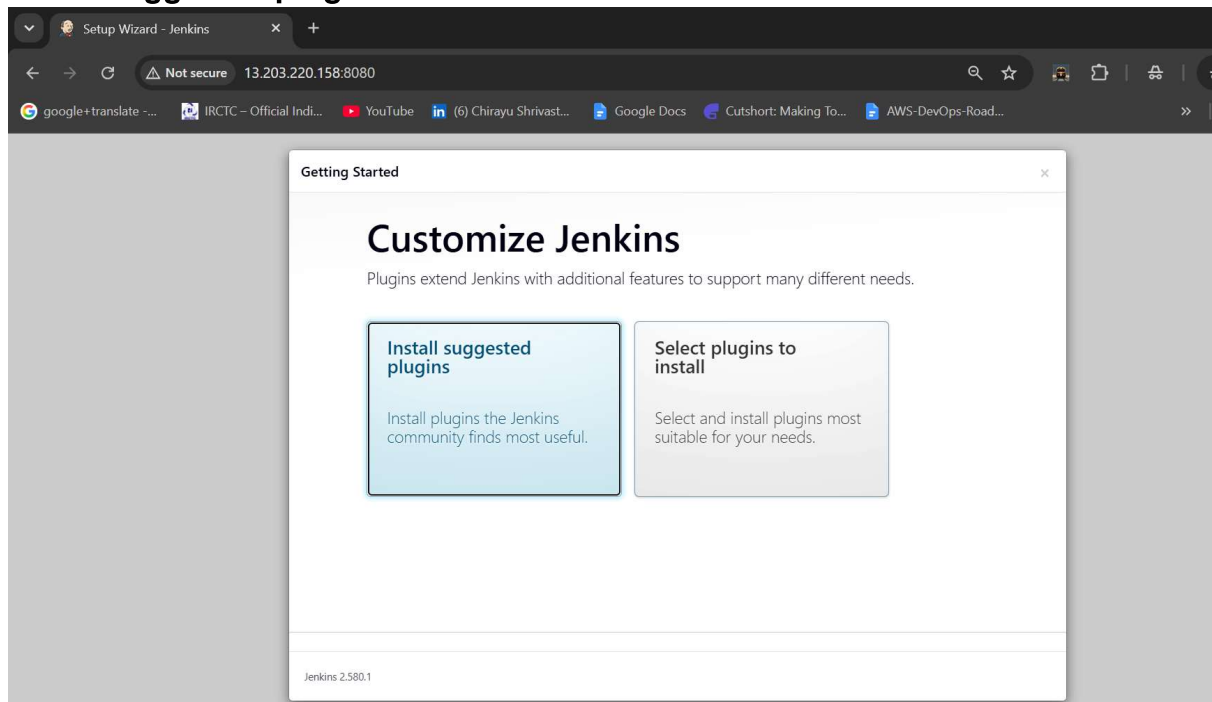
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]>
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> *****
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> *****
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> *****
Oct 08 07:33:46 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:46.901+0000 [id=31] INFO jenkins.InitReactorRunner$1onAttained: Completed initialization
Oct 08 07:33:46 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:46.936+0000 [id=26] INFO hudson.lifecycle.Lifecycle$onReady: Jenkins is fully up and running
Oct 08 07:33:46 ip-172-31-2-191 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 08 07:33:48 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:48.873+0000 [id=50] INFO h.m.DownloadService$Downloadable$Load: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller
Oct 08 07:33:48 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:48.875+0000 [id=50] INFO hudson.util.Retrier$start: Performed the action check updates server successfully at the attempt #1
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu$
```

CI/CD Pipelines (Jenkins):

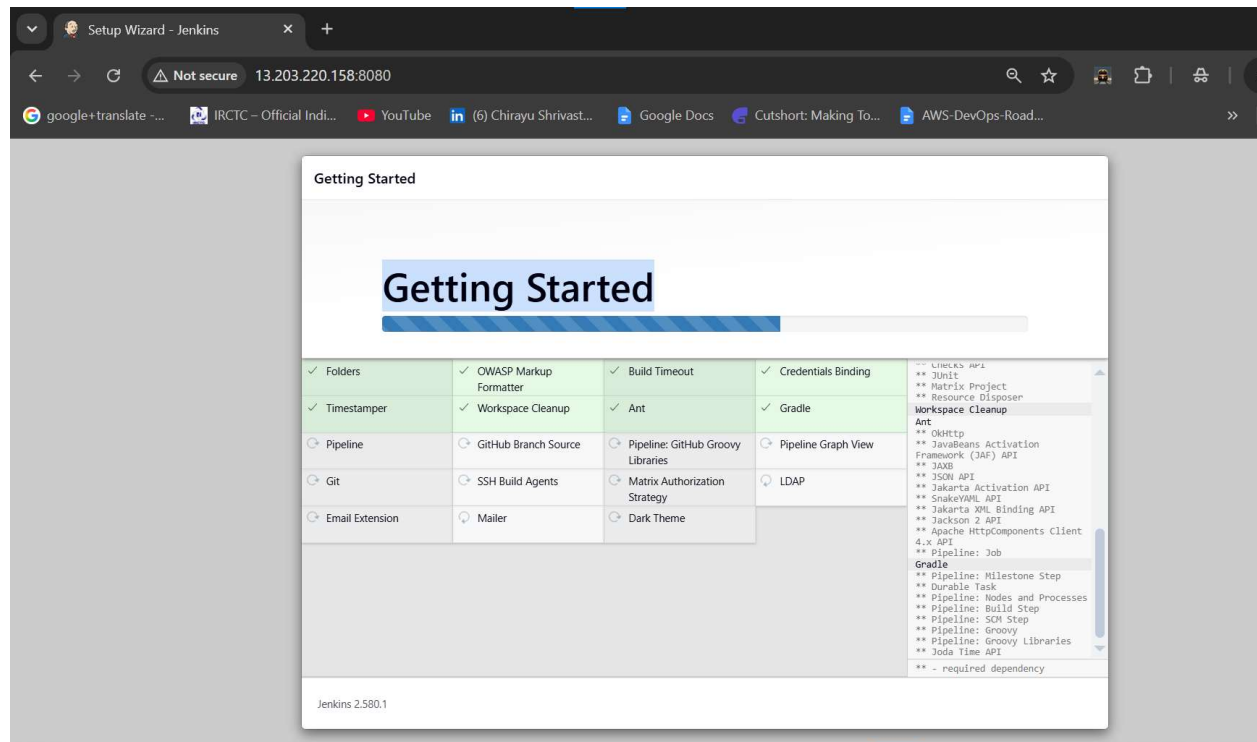
sudo cat /var/lib/jenkins/secrets/initialAdminPassword:



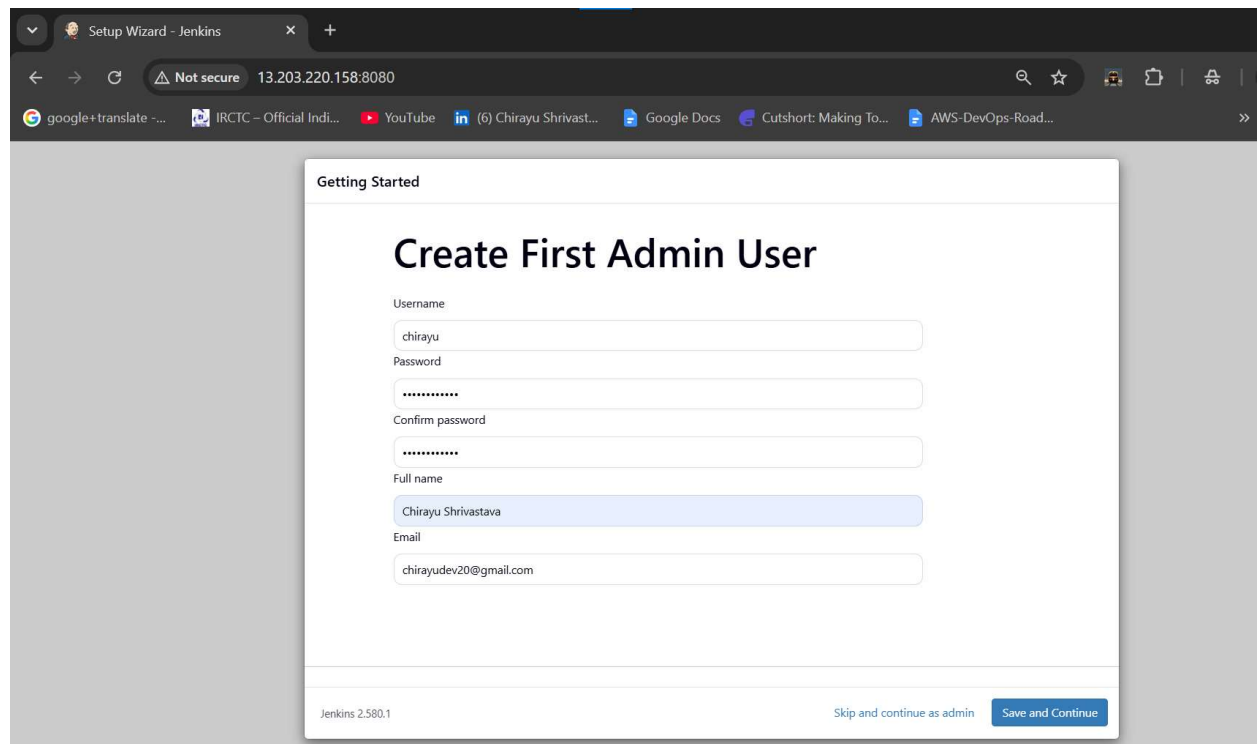
Install suggested plugins :

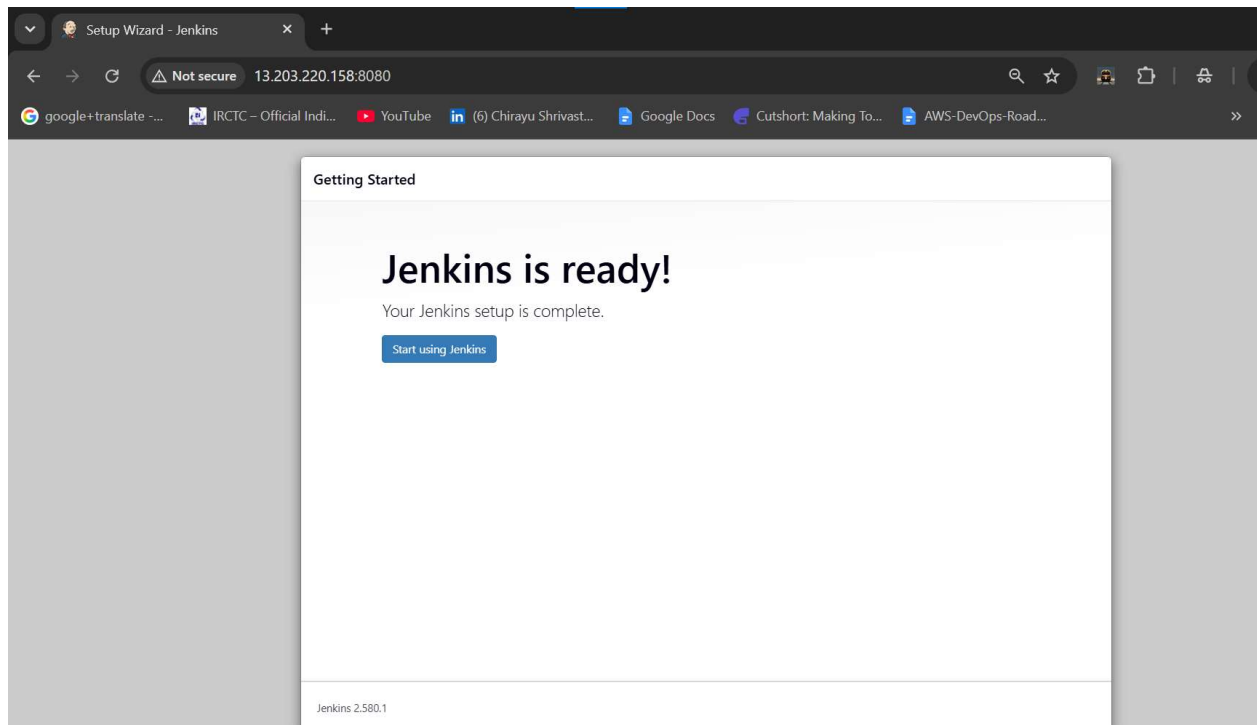


Getting Started:

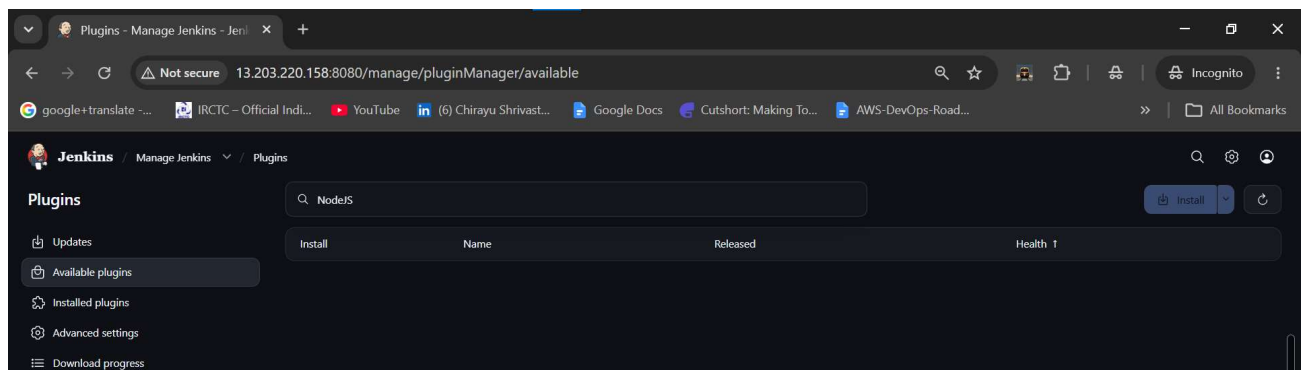


Create First Admin User:



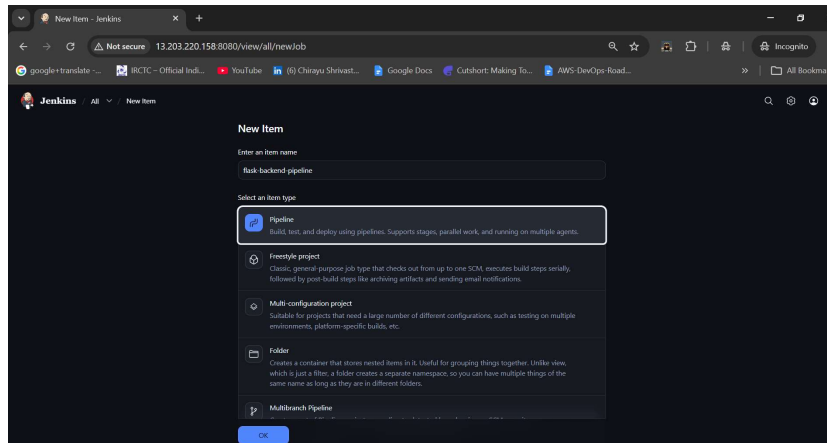


NodeJS Plugin Check & Install :

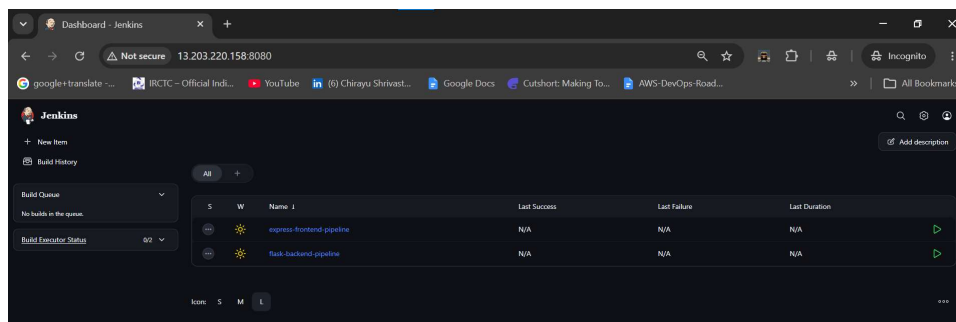
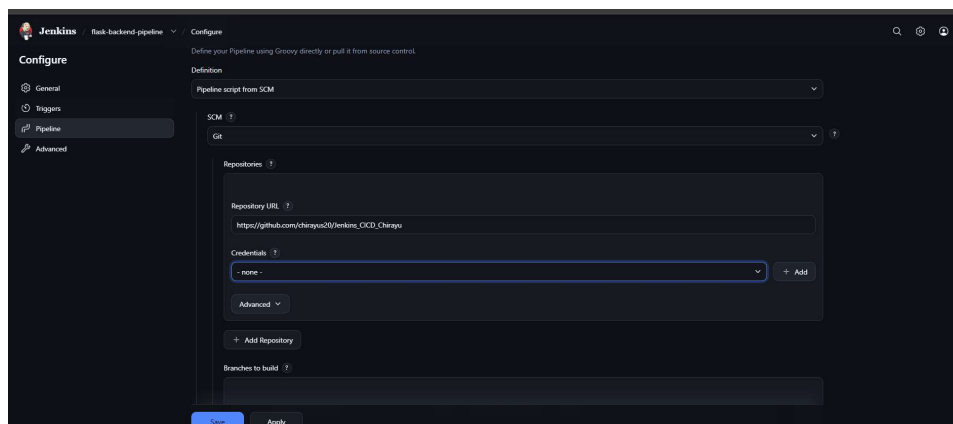


Step 4: Create the Backend Frontend Pipeline Job

1. On the Jenkins dashboard, click New Item.
2. Name it **flask-backend-pipeline** and **flask-frontend-pipeline**
3. Select Pipeline and click OK.



Configure Git in Jenkins.



GitHub Webhook Setup

EC2 | ap-south-1

Add webhook

github.com/chirayus20/Jenkins_CICD_Chirayu/settings/hooks/new

YouTube Google Translate Google Account F5 NGINX Product... Docker Hub Contai... Log in | MongoDB AWS_Devops_Interv... IRCTC - Official Indi...

General

Access

Collaborators

Moderation

Code, planning, and automation

Rulesets

Branches

Tags

Actions

Webhooks

Copilot

Planning

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

http://13.203.220.158:8080/github-webhook/

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

☐ Let me select individual events.

☒ Active

We will deliver event details when this hook is triggered.

Add webhook

EC2 | ap-south-1

Webhooks - Settings - chirayus20

github.com/chirayus20/Jenkins_CICD_Chirayu/settings/hooks

YouTube Google Translate Google Account F5 NGINX Product... Docker Hub Contai... Log in | MongoDB AWS_Devops_Interv... IRCTC - Official Indi...

chirayus20 / Jenkins_CICD_Chirayu

Type to search

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Okay, that hook was successfully created. We sent a ping payload to test it out! Read more about it at <https://docs.github.com/webhooks/#ping-event>.

General

Access

Collaborators

Moderation

Code, planning, and automation

Rulesets

Branches

Tags

Webhooks

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).

☒ http://13.203.220.158:8080/github-webhook/ (push)

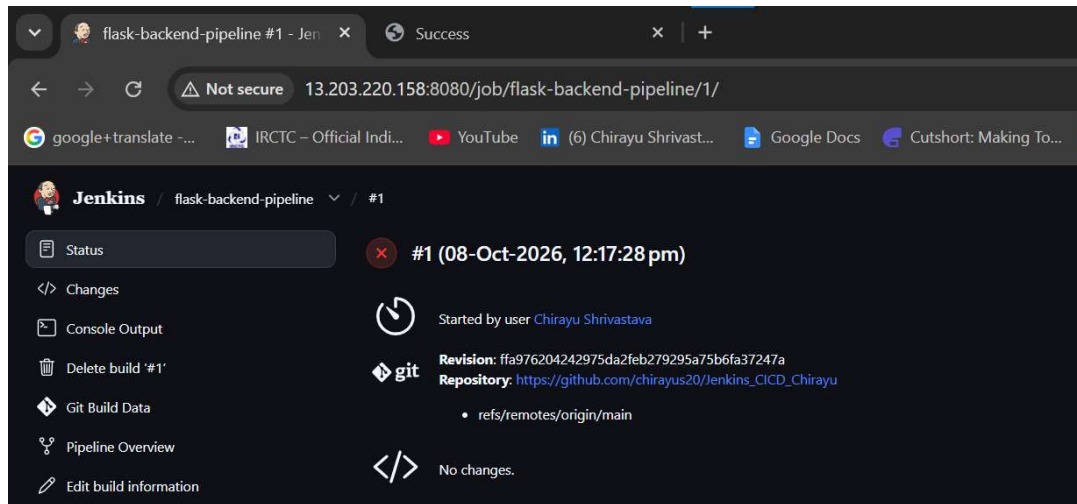
Last delivery was successful.

Edit Delete

Add webhook

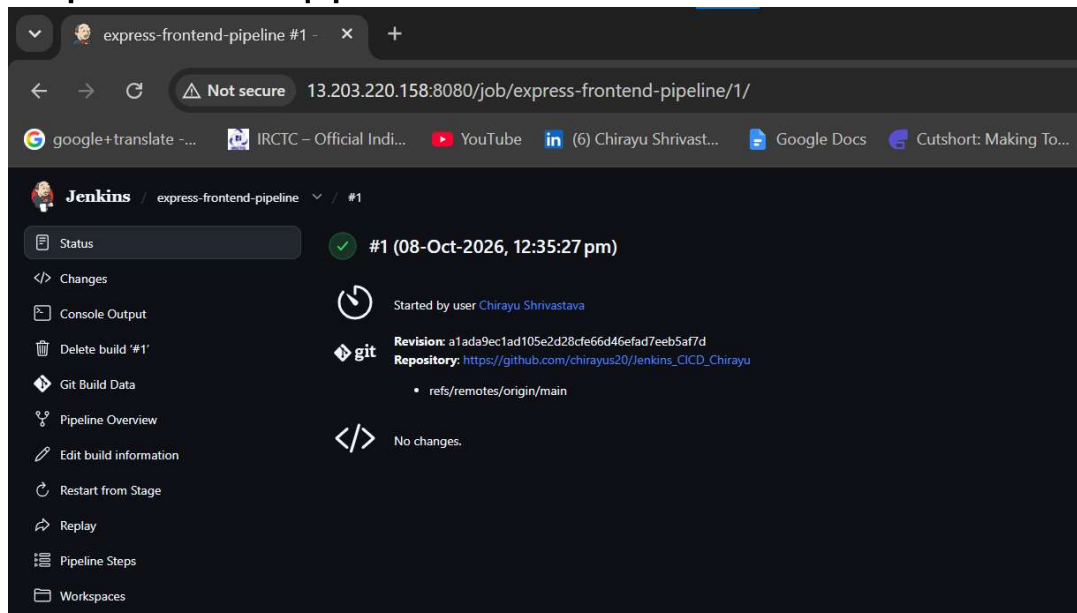
Step 5: Run the First Build Manually (Verification)

flask-backend-pipeline



The screenshot shows the Jenkins web interface for the 'flask-backend-pipeline' job, build #1. The browser address bar shows the URL '13.203.220.158:8080/job/flask-backend-pipeline/1/'. The Jenkins header shows the job name 'flask-backend-pipeline' and build number '#1'. The build status is 'Success' (indicated by a green checkmark). The build was started by user 'Chirayu Shrivastava' on 08-Oct-2026 at 12:17:28 pm. The build is linked to a Git repository: 'https://github.com/chirayus20/Jenkins_CICD_Chirayu', revision 'ffa976204242975da2feb279295a75b6fa37247a', and branch 'refs/remotes/origin/main'. The build has no changes. The left sidebar contains links to 'Status', 'Changes', 'Console Output', 'Delete build #1', 'Git Build Data', 'Pipeline Overview', and 'Edit build information'.

express-frontend-pipeline



The screenshot shows the Jenkins web interface for the 'express-frontend-pipeline' job, build #1. The browser address bar shows the URL '13.203.220.158:8080/job/express-frontend-pipeline/1/'. The Jenkins header shows the job name 'express-frontend-pipeline' and build number '#1'. The build status is 'Success' (indicated by a green checkmark). The build was started by user 'Chirayu Shrivastava' on 08-Oct-2026 at 12:35:27 pm. The build is linked to a Git repository: 'https://github.com/chirayus20/Jenkins_CICD_Chirayu', revision 'a1ada9ec1ad105e2d28cfe66d46efad7eeb5af7d', and branch 'refs/remotes/origin/main'. The build has no changes. The left sidebar contains links to 'Status', 'Changes', 'Console Output', 'Delete build #1', 'Git Build Data', 'Pipeline Overview', 'Edit build information', 'Restart from Stage', 'Replay', 'Pipeline Steps', and 'Workspaces'.

Restarting the Backend and Frontend with PM2

If a service shows errored in PM2, clear its old logs, remove it, and start it again.

Backend (Flask)

commands:

```
cd ~/Jenkins_CICD_Chirayu/backend
```

```
pm2 flush backend
```

```
pm2 delete backend
```

```
pm2 start app.py --name backend --interpreter ./myenv/bin/python3
```

```
sleep 5
```

```
pm2 status
```

Frontend (Express)

commands:

```
cd ~/Jenkins_CICD_Chirayu/frontend
```

```
npm install
```

```
pm2 flush express-frontend
```

```
pm2 delete express-frontend
```

```
pm2 start server.js --name express-frontend
```

```
sleep 3
```

```
pm2 status
```

Backend and frontend are online in PM2. Test data is stored in MongoDB successfully.

```
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu/frontend$ sleep 3
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu/frontend$ pm2 status
```

id	name	namespace	version	mode	pid	uptime	u	status	cpu	mem	user	watching
0	backend	default	N/A	fork	42223	16m	0	online	0%	42.4mb	ubuntu	disabled
2	express-frontend	default	1.0.0	fork	42585	14s	0	online	0%	53.9mb	ubuntu	disabled

```
host metrics | cpu: 2.2% | ram usage: 84.4% | ens5: ↓ 0.002mb/s ↑ 0.001mb/s | disk: ↓ 0.249mb/s ↑ 0.405mb/s
```

Dashboard - Jenkins

Not secure 13.203.220.158:8080

Jenkins

+ New Item

Build History

Build Queue

Build Executor Status 0/2

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	express-frontend-pipeline	7 hr 0 min #1	N/A	14 sec
✓	☁	flask-backend-pipeline	7 hr 3 min #2	7 hr 18 min #1	15 sec

All - Jenkins

Not secure 13.203.220.158:8080/view/all/builds

Jenkins / All / Build History

+ New Item

Build History

Build Queue

Build Executor Status 0/2

Build History of Jenkins

S	Build	Time Since ↑	Status
✓	express-frontend-pipeline #1	6 hr 21 min	stable
✓	flask-backend-pipeline #2	6 hr 24 min	back to normal
✗	flask-backend-pipeline #1	6 hr 39 min	broken since this build

Icon: S M L

Dashboard - Jenkins

Submit Form

Not secure 13.203.220.158:8000

Submit Form

Name

Chirayu-test2

Password

Submit

Dashboard - Jenkins

Success

Not secure 13.203.220.158:8000/success.html

Success!

Data submitted successfully.

Go Back

ORGANIZATION Chirayu's Org - 2025-...

PROJECT Project0

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

clusters

admin

new-mongo-db

Data

load

new-mongo-db > data

Cluster0 > new-mongo-db > data

Documents Aggregations Schema Indexes Validation

Type a query: (field: 'value') or **Generate Query**

Explain Reset Find Options

ADD DATA SCALE EXPORT CODE

```
{
  "_id": "8d9c15d171bc7f93f8f723b1c3608b37",
  "name": "Chirayu-test2",
  "password": "94c7gk1727681d1154c8u0w3z3uq05082w98b43b7085ue7f0a3a7xvc37ad209d2977043d4b737f3u8f725a088a077f0f743cc550u0e4b4b70d7b123777d47f92508b2..."
}
```